

Advanced Example: Map-Reduce with Callbacks

Let's implement a simple Map-Reduce pattern using callbacks:

```
#include <stdio.h>
#include <stdlib.h>

// Map function: applies a transformation to each element in an array
int* mapArray(int arr[], int size, int (*transform)(int)) {
    int* result = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        result[i] = transform(arr[i]);
    }
    return result;
}

// Reduce function: reduces an array to a single value using an accumulator
int reduceArray(int arr[], int size, int (*combine)(int, int), int
initialValue) {
    int accumulator = initialValue;
    for (int i = 0; i < size; i++) {
        accumulator = combine(accumulator, arr[i]);
    }
    return accumulator;
}

// Transform functions for map
int square(int x) {
    return x * x;
}

int double_value(int x) {
    return x * 2;
}

int absolute(int x) {
    return x < 0 ? -x : x;
}

// Combine functions for reduce
int sum(int acc, int value) {
    return acc + value;
}

int product(int acc, int value) {
    return acc * value;
}

int max(int acc, int value) {
    return acc > value ? acc : value;
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```

int main() {
    int arr[] = {-3, 2, 0, 7, -5, 11, 4};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, size);

    // Map examples
    int* squares = mapArray(arr, size, square);
    printf("Squares: ");
    printArray(squares, size);

    int* doubles = mapArray(arr, size, double_value);
    printf("Doubled values: ");
    printArray(doubles, size);

    int* absolutes = mapArray(arr, size, absolute);
    printf("Absolute values: ");
    printArray(absolutes, size);

    // Reduce examples
    printf("Sum of elements: %d\n", reduceArray(arr, size, sum, 0));
    printf("Product of elements: %d\n", reduceArray(arr, size, product,
1));
    printf("Maximum element: %d\n", reduceArray(arr, size, max, arr[0]));

    // Combine map and reduce
    printf("Sum of squares: %d\n", reduceArray(squares, size, sum, 0));
    printf("Sum of absolute values: %d\n", reduceArray(absolutes, size,
sum, 0));

    // Free allocated memory
    free(squares);
    free(doubles);
    free(absolutes);

    return 0;
}

```

Output:

```

Original array: -3 2 0 7 -5 11 4
Squares: 9 4 0 49 25 121 16
Doubled values: -6 4 0 14 -10 22 8
Absolute values: 3 2 0 7 5 11 4
Sum of elements: 16
Product of elements: 0
Maximum element: 11
Sum of squares: 224
Sum of absolute values: 32

```

Custom Data Processing Example

This example shows how to use callbacks for creating custom data processing pipelines:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

// Define a structure for a person
typedef struct {
    char name[50];
    int age;
    float height; // in meters
    float weight; // in kg
} Person;

// Calculate BMI (Body Mass Index)
float calculateBMI(Person person) {
    return person.weight / (person.height * person.height);
}

// Filter callback type for people
typedef int (*PersonFilter)(Person);

// Map callback type for people
typedef void (*PersonTransform)(Person*);

// Print callback
typedef void (*PersonPrinter)(Person);

// Count people that match a condition
int countPeople(Person people[], int size, PersonFilter filter) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (filter(people[i])) {
            count++;
        }
    }
    return count;
}

// Filter people into a new array
Person* filterPeople(Person people[], int size, PersonFilter filter, int*
resultSize) {
    *resultSize = countPeople(people, size, filter);
    Person* result = (Person*)malloc(*resultSize * sizeof(Person));

    int j = 0;
    for (int i = 0; i < size; i++) {
        if (filter(people[i])) {
            result[j++] = people[i];
        }
    }

    return result;
}

// Transform each person in the array
void transformPeople(Person people[], int size, PersonTransform transform)
{
    for (int i = 0; i < size; i++) {
        transform(&people[i]);
    }
}

// Print people using a custom printer
void printPeople(Person people[], int size, PersonPrinter printer) {
    for (int i = 0; i < size; i++) {
        printer(people[i]);
    }
}

```

```

    }
}

// Filter functions
int isAdult(Person p) {
    return p.age >= 18;
}

int isOverweight(Person p) {
    // BMI over a threshold
    float bmi = calculateBMI(p);
    return bmi >= 25.0;
}

int isTall(Person p) {
    return p.height >= 1.8; // 1.8m or taller
}

// Transform functions
void adjustWeight(Person* p) {
    // Simulate weight change
    p->weight = p->weight * 0.95; // 5% loss
}

void growOlder(Person* p) {
    p->age++;
}

// Print functions
void printBasic(Person p) {
    printf("Name: %s, Age: %d\n", p.name, p.age);
}

void printDetailed(Person p) {
    printf("Name: %s, Age: %d, Height: %.2fm, Weight: %.1fkg, BMI: %.1f\n",
        p.name, p.age, p.height, p.weight, calculateBMI(p));
}

int main() {
    // Sample data
    Person people[] = {
        {"Alice", 25, 1.65, 55.0},
        {"Bob", 17, 1.82, 70.0},
        {"Charlie", 42, 1.78, 95.0},
        {"David", 30, 1.90, 85.0},
        {"Eve", 22, 1.60, 50.0}
    };
    int size = sizeof(people) / sizeof(people[0]);

    printf("All people (basic info):\n");
    printPeople(people, size, printBasic);

    printf("\nAll people (detailed info):\n");
    printPeople(people, size, printDetailed);

    // Filter examples
    int adultCount;
    Person* adults = filterPeople(people, size, isAdult, &adultCount);
    printf("\nAdults only:\n");
    printPeople(adults, adultCount, printDetailed);
}

```

```

    int tallCount;
    Person* tallPeople = filterPeople(people, size, isTall, &tallCount);
    printf("\nTall people only:\n");
    printPeople(tallPeople, tallCount, printDetailed);

    // Transform example
    printf("\nAfter weight adjustment:\n");
    transformPeople(people, size, adjustWeight);
    printPeople(people, size, printDetailed);

    // Free allocated memory
    free(adults);
    free(tallPeople);

    return 0;
}

```

Output:

All people (basic info):

```

Name: Alice, Age: 25
Name: Bob, Age: 17
Name: Charlie, Age: 42
Name: David, Age: 30
Name: Eve, Age: 22

```

All people (detailed info):

```

Name: Alice, Age: 25, Height: 1.65m, Weight: 55.0kg, BMI: 20.2
Name: Bob, Age: 17, Height: 1.82m, Weight: 70.0kg, BMI: 21.1
Name: Charlie, Age: 42, Height: 1.78m, Weight: 95.0kg, BMI: 30.0
Name: David, Age: 30, Height: 1.90m, Weight: 85.0kg, BMI: 23.5
Name: Eve, Age: 22, Height: 1.60m, Weight: 50.0kg, BMI: 19.5

```

Adults only:

```

Name: Alice, Age: 25, Height: 1.65m, Weight: 55.0kg, BMI: 20.2
Name: Charlie, Age: 42, Height: 1.78m, Weight: 95.0kg, BMI: 30.0
Name: David, Age: 30, Height: 1.90m, Weight: 85.0kg, BMI: 23.5
Name: Eve, Age: 22, Height: 1.60m, Weight: 50.0kg, BMI: 19.5

```

Tall people only:

```

Name: Bob, Age: 17, Height: 1.82m, Weight: 70.0kg, BMI: 21.1
Name: David, Age: 30, Height: 1.90m, Weight: 85.0kg, BMI: 23.5

```

After weight adjustment:

```

Name: Alice, Age: 25, Height: 1.65m, Weight: 52.2kg, BMI: 19.2
Name: Bob, Age: 17, Height: 1.82m, Weight: 66.5kg, BMI: 20.1
Name: Charlie, Age: 42, Height: 1.78m, Weight: 90.2kg, BMI: 28.5
Name: David, Age: 30, Height: 1.90m, Weight: 80.8kg, BMI: 22.4
Name: Eve, Age: 22, Height: 1.60m, Weight: 47.5kg, BMI: 18.6

```

Common Pitfalls and Best Practices

Recursion Pitfalls

1. **Stack Overflow:** Recursive functions can cause stack overflow if there are too many recursive calls or if the base case is not properly defined.
2. // Dangerous recursion - may cause stack overflow
3.

```
int dangerous_recursion(int n) {
```

```

4.     // Missing or incorrect base case
5.     return n + dangerous_recursion(n - 1);
6. }

```

7. **Inefficient Repeated Calculations:** As seen in the naive Fibonacci implementation, recursion can lead to many redundant calculations.

8. **Memory Usage:** Each recursive call adds a new layer to the call stack, consuming memory.

Best Practices for Recursion

1. **Always Define a Proper Base Case:** Ensure there's always a way to terminate the recursion.

```

2. int safe_recursion(int n) {
3.     // Proper base case
4.     if (n <= 0) {
5.         return 0;
6.     }
7.     return n + safe_recursion(n - 1);
8. }

```

9. **Consider Tail Recursion:** Tail recursion can be optimized by many compilers.

```

10. // Tail-recursive factorial function
11. unsigned long factorial_tail(unsigned int n, unsigned long
    accumulator) {
12.     if (n == 0 || n == 1) {
13.         return accumulator;
14.     }
15.     return factorial_tail(n - 1, n * accumulator);
16. }
17.
18. // Wrapper function
19. unsigned long factorial(unsigned int n) {
20.     return factorial_tail(n, 1);
21. }

```

22. **Use Memoization for Expensive Calculations:** Store results of subproblems to avoid redundant calculations.

```

23. #include <stdio.h>
24. #include <stdlib.h>
25.
26. // Fibonacci with memoization
27. int fibonacci_memo(int n, int* memo) {
28.     // Base cases
29.     if (n <= 0) return 0;
30.     if (n == 1) return 1;
31.
32.     // Check if already computed
33.     if (memo[n] != -1) {
34.         return memo[n];
35.     }
36.
37.     // Compute and store
38.     memo[n] = fibonacci_memo(n - 1, memo) + fibonacci_memo(n - 2,
        memo);
39.     return memo[n];
40. }
41.
42. int fibonacci(int n) {
43.     // Initialize memoization array
44.     int* memo = (int*)malloc((n + 1) * sizeof(int));

```

```

45.     for (int i = 0; i <= n; i++) {
46.         memo[i] = -1;
47.     }
48.
49.     int result = fibonacci_memo(n, memo);
50.     free(memo);
51.     return result;
52. }

```

53. **Consider Iterative Alternatives:** For some problems, an iterative solution might be more efficient.

Callback Best Practices

1. **Type Safety:** Use typedefs for callback function pointers to improve clarity and safety.

```

2. typedef int (*ComparisonFunction)(int, int);
3.
4. void bubbleSort(int arr[], int size, ComparisonFunction compare) {
5.     // Implementation
6. }

```

7. **Error Handling:** Check if callback functions are valid before using them.

```

8. void safeFunction(int arr[], int size, int (*callback)(int)) {
9.     if (callback == NULL) {
10.         printf("Error: Null callback function\n");
11.         return;
12.     }
13.     // Use callback
14. }

```

15. **Context Data:** Pass additional context data to callbacks when needed.

```

16. typedef struct {
17.     // Context data
18.     int threshold;
19.     int multiplier;
20. } Context;
21.
22. int transformWithContext(int value, Context* ctx) {
23.     if (value > ctx->threshold) {
24.         return value * ctx->multiplier;
25.     }
26.     return value;
27. }
28.
29. void processWithContext(int arr[], int size, int (*transform)(int,
Context*), Context* ctx) {
30.     for (int i = 0; i < size; i++) {
31.         arr[i] = transform(arr[i], ctx);
32.     }
33. }

```

Conclusion

This document has covered fundamental concepts in C programming related to recursion, searching algorithms, sorting algorithms, and the use of callbacks for problem-solving. These techniques form the building blocks of many advanced algorithms and data structures.

Recursion offers an elegant way to solve problems by breaking them down into smaller instances of the same problem. While it can lead to cleaner code, developers must be mindful of its memory and performance implications.

Searching and sorting algorithms are essential for efficient data manipulation, and understanding their recursive implementations helps in grasping their underlying principles.

Callbacks provide a powerful mechanism for writing flexible and reusable code in C. They allow passing behavior as arguments, enabling generic functions that can be customized for specific needs.

By combining these techniques, C programmers can create efficient, maintainable, and flexible code to solve a wide variety of problems.